

Trabalho Prático 3 – Máquina de Busca Avançada

Disciplina Estrutura de Dados

Raphaela Maria Costa e Silva - 2020006973

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG – Brasil

rapha470@gmail.com

1. Introdução

A proposta do TP foi implementar um programa que busca palavras em um conjunto de arquivos (corpus) e retorna os dez arquivos com melhores classificações. Foi realizada uma implementação em duas partes: primeiro a criação do índice invertido e depois a consulta.

Essa documentação tem como objetivo explicar o problema de maneira sucinta, explicar as estruturas de dados e os métodos usados para solucionar o problema, analisar a complexidade de tempo e de espaço do código e apresentar os resultados de análises experimentais em relação ao desempenho computacional e à localidade de referência.

Como solicitado no enunciado do TP, foi usado o método de hashing para criar o índice invertido. Os dados a serem pesquisados são passados por arquivos de texto localizados em um diretório, que é passado como parâmetro na linha de comando. O programa também gera os dados para a análise experimental a partir da biblioteca ‘memlog’, e os erros do programa são tratados com as macros definidas em ‘msgassert.h’.

2. Implementação

A implementação do programa teve como base as estruturas de fila Índice e Vocabulário.

O Índice foi implementado com célula início (Chave), célula fim (Chave), e tamanho (int). A implementação da célula início ou célula cabeça reduz o custo assintótico de algumas funções e facilita a iteração dentro da fila. A célula fim é útil para definir casos de parada em loops. Já o tamanho evita a necessidade de iterar a lista inteira para achar a quantidade de membros da fila.

Já a célula (no código é a estrutura Chave), possui palavra (char*), qtd (int), nome_doc(int), uma estrutura de ponteiro para o próximo elemento do Índice. A palavra guarda o nome da palavra, o qtd representa quantas vezes a palavra apareceu no documento e o nome_doc é o id do documento. Já a estrutura que aponta para o próximo elemento da Fila é necessária para conectar as Chaves no Índice e iterar pelo Índice. Isso torna a estrutura dinâmica, e foi feito dessa forma pois não era possível prever a quantidade de chaves teriam na fila.

Essa estrutura possui as seguintes funções:

Void inicialização (Índice) – Inicializa o Índice com valores nulos.

int remover (Índice * sequencia) – remove todos os elementos do Índice.

Inserir - cria um elemento Chave e insere no final da fila.

O Vocabulario foi implementado com célula início (Palavra), célula fim (Palavra), e tam (int). A implementação da célula início ou célula cabeça reduz o custo assintótico de algumas funções e facilita a iteração dentro da fila. A célula fim ajuda na inserção de novos elementos, visto que com uma célula fim não é necessário iterar toda a fila para encontrar o último elemento. Já o tamanho evita a necessidade de iterar a lista inteira para achar a quantidade de membros da fila.

Já a célula (no código é a estrutura Palavra), possui word (char*), qtd (int), uma estrutura de ponteiro para a próxima Palavra. Word é a palavra do Vocabulario, a qtd é a quantidade de vezes que aquela palavra aparece. Já a estrutura que aponta para o próximo elemento da Lista é necessária para conectar as Palavra's na Lista e iterar pela Lista. Isso torna a estrutura dinâmica, e foi feito dessa forma pois não era possível prever a quantidade de Palavras teriam em cada lista.

As funções das duas estruturas acima são:

void criar_lista(Vocabulario *lista) – cria a lista com valores Null;

void remover(Vocabulario *lista) – remove todos elementos do Vocabulario;

Inserir no início - insere Palavra na posição de início da lista.

Inserir ordenado - insere no início.

Não duplicado – itera a lista e se a Palavra já existir retorna falso e atualiza a quantidade de vezes que a palavra aparece, se não retorna true.

Já na função main(int argc, char ** argv), primeiro são coletados da linha de comando o nome do arquivo da consulta, o nome do arquivo de saída, o nome do arquivo que contém as stopwords, e o nome do diretório que contém o corpus, o arquivo para desempenho computacional e a opção de ativar o registro de acesso. Isso é feito através da função parse_args(int argc, char ** argv).

No main foram feitos dois loops principais:

No primeiro loop é lido todos os arquivos do corpus. O conteúdo lido de cada arquivo é tratado colocando em lower case e trocando os caracteres que não pertencem à tabela ASCII (ou que são menores que 96, como números) por espaço. Então o conteúdo gerado é colocado em outro arquivo “vocabulário”. Então o arquivo vocabulário é lido string por string, e as strings são inseridas na estrutura Vocabulario (variável lista). A variável lista é feita para cada arquivo. A inserção é feita sem inserir palavras repetidas, no caso da palavra ser repetida a quantidade que ela aparece é atualizada.

Em seguida, para cada palavra inserida na lista é feito seu hashing e ela é ou descartada por ser uma stopword ou é inserida no índice junto com sua frequência naquele arquivo e seu id. Foram consideradas as stopwords passadas pelo professor no Moodle, portanto, elas possuem um tamanho fixo de no máximo 127 palavras com no máximo 20 caracteres cada.

Para palavras muito grandes com hashing acima de 10000, essas palavras foram colocadas em no índice 10001 do Índice, para o índice não ficar grande ao extremo.

O hashing é feito pela função `hashing(std::string)`, a função percorre toda string passada por parâmetro e faz a soma de cada termo usando a tabela ASCII e peso para cada posição da string.

Finalmente é calculado o `wd`, que é a raiz quadrada da quantidade de termos do documento. O `wd` é guardado em um array de `double` em que o índice corresponde ao `id` do documento.

Já no segundo loop para cada termo no arquivo da consulta, é feito o cálculo para decidir os melhores resultados. Então é verificado se o termo da consulta está no índice usando hashing, se não está esse termo não é calculado. Se o termo consta no índice, é verificada a lista de documentos que possuem o termo e é calculado o `idf` e o `Wq` da consulta. O `idf` é calculado pelo log natural do tamanho do corpus dividido pela quantidade de documentos com o termo consultado, já o `Wq` é a soma do quadrado das `idfs` de cada termo da consulta.

Então para cada documento que possui o termo é calculado o resultado. Primeiro é calculado o `wdt` de cada documento, que é a frequência do termo vezes o `idf`. Em seguida, o resultado é a soma da multiplicação do `wqt` com o `wdt` de cada termo na consulta. O resultado de cada documento é normalizado dividindo o resultado calculado por `wq` vezes o `wd` (calculado anteriormente) daquele arquivo, e esse resultado é passado para a variável imprimir junto ao `id` do documento.

Finalmente, para imprimir no arquivo de saída é feito um loop escolhendo os dez documentos com maior resultado, em caso de empate documento com menor `id` foi priorizado, isso ocorre pois o array que possui os resultados está em ordem crescente e o método de ordenação é estável.

3. Estudo de Complexidade

Tempo:

Void `criar_lista` – complexidade $O(1)$.

Void `inserir_no_inicio` – complexidade $O(1)$.

Void `inserir_ordenado` – complexidade $O(n)$.

Bool `não_duplicado` – complexidade $O(n)$.

Int `hashing` – complexidade $O(n)$.

Void `inicialização` – complexidade $O(1)$.

Void `inserir` – complexidade $O(1)$.

Int `remover` – $O(n)$ pior caso, $O(1)$ melhor caso (quando a lista tem um elemento).

No `main` é possível ver que a complexidade do código depende do tamanho do corpus.

No começo do código é recebido as stopwords, que é um custo $O(1)$, pois é predefinido que são 127 stopwords. E então é inicializado todas as posições do Índice, isso também é $O(1)$, pois inicializar é $O(1)$ e o tamanho do índice é predefinido em 10000.

No primeiro loop principal, todo o diretório do corpus é percorrido e então todas as palavras de cada documento é tratada para então ou ser descartada ou ser inserida no vocabulário, e então as palavras no vocabulário são inseridas no índice e são removidas do Vocabulário. Isso faz com que inserir no vocabulário, inserir no índice, tratar os caracteres indesejados e remover sejam $O(n^2)$.

Já no segundo loop é necessário percorrer somente uma lista do índice algumas vezes para cada termo pesquisado. Dessa forma, o custo assintótico no pior caso do segundo loop principal é $O(n^2)$, que seria o caso de ter tantos termos pesquisados quanto documentos no corpus, mas geralmente são poucos termos pesquisados, então a complexidade é $O(n)$. Portanto, o primeiro loop é muito mais significativo em termos de complexidade assintótica do que o segundo loop, pois o primeiro loop precisa percorrer todos os termos do corpus, enquanto o segundo loop percorre uma lista.

Por fim são removidos da memória os elementos do Índice, isso é $O(n)$, pois foi predefinido que o Índice não teria um hashing maior que 10000. Então para os 10000 indexes do Índice é removido uma lista, como já visto, remover custa $O(1)$, portanto: $O(1) * O(n) = O(n)$.

Portanto, é perceptível que o primeiro loop principal tem um peso assintótico maior, fazendo o código ser $O(n^2)$.

Espaço:

No código, além de outras variáveis que ocupam $O(1)$ de espaço, foram usadas uma array do tipo Índice uma variável do tipo Vocabulário e um arquivo para guardar a string modificada. Todas essas variáveis e estruturas usadas dependem do tamanho de cada documento passado na entrada, enquanto o Índice depende do tamanho de todo o corpus. Portanto, é possível afirmar que o código tem complexidade de espaço $O(n)$.

4. Estratégias de Robustez

A robustez do programa foi implementada a partir do arquivo 'msgassert.h' usando asserções aplicadas ao código considerando uma condição e uma mensagem de erro.

As condições que levam ao erro são:

Se o arquivo com a consulta, o arquivo de saída, o arquivo de stopwords ou o diretório do corpus não for definido, ou seja, o arquivo passado na linha de comando for inválido.

Se o arquivo do registro de acesso de tempo e memória não for definido, ou seja, o arquivo passado na linha de comando for inválido.

E se haver algum problema com a alocação de memória para o tipo Palavra ou Chave.

Caso algum dos caracteres da consulta for invalido, ou seja, tiver um valor menor do que 97 na tabela ASCII.

5. Testes

6. Análise Experimental

Desempenho Computacional

Análise de Padrão de Acesso à Memória e Localidade de Referência

7. Conclusões

A proposta desse trabalho foi realizar a implementação de uma máquina avançada de busca. Foi um desafio fazer esse trabalho e implementar o hashing. Aprendi sobre como implementar o hashing e como fazer um heap. O trabalho também trouxe diversos conhecimentos sobre manipulação de arquivos em diretórios diferentes.

Também adquiri conhecimentos sobre algoritmos de busca, curiosidades sobre o uso do espaço vetorial no cálculo da busca, sobre o índice invertido e o tf idf.

8. Bibliografia

CHAIMOWICS, L and Prates. Slides virtuais e vídeos da disciplina de estrutura de dados. Disponibilizados via moodle na seção Algoritmos de Pesquisa (Hashing).

As estruturas Índice e Vocabulário e suas funções foram tiradas do TP1 feito por mim.

9. Instruções para compilação e execução

O programa foi desenvolvido na linguagem C++, compilada por G++ da GNU Compiler Collection no SO Linux.

Acesse o diretório TP. Compile os arquivos usando o comando make. O executável gerado estará em bin. Agora entre no diretório bin e execute o programa.

Para executar o programa deve-se passar na linha de comando “-i” seguido do arquivo da consulta, “-o” seguido do arquivo de saída, “-c” seguido do nome do diretório do corpus, “-s” seguido do arquivo de stopwords, “-p” seguido do nome do arquivo onde será feito o registro de acesso. A última opção de argumento na linha de parâmetro é facultativa, escreva ‘l’ caso desejar que seja registrado os acessos à memória, não escreva nada caso quiser que seja registrado somente o tempo de execução.

Os arquivos passados pela linha de comando devem estar no diretório bin e o diretório do corpus deve estar dentro do diretório TP.

Por exemplo: ” ./buscar -i consulta.txt -o ranking.txt -c corpus -s stopwords -p analise.txt -l”

O arquivo de saída seria o “ranking.txt” e será registrado os acessos à memória no arquivo analise.txt. A consulta estaria no arquivo consulta.txt. Os arquivos do corpus estariam no diretório corpus e as stopwords no arquivo stopwords.